Collection ToTAL « Réconciliation » | Guide pratique #2

Guide Jaune

FORGE : PLANIFIER + IMPLÉMENTER

Transformer les spécifications en livraisons maîtrisées

Livre 17 · Hémisphère DELIVERY · Section FORGE du SDLC réconcilié

Léo Michel Bélanger

« FORGE ne construit pas ce qui a été demandé. FORGE construit ce qui a été compris, spécifié, et rendu testable. »

Comment utiliser ce guide

Le **Guide Jaune** couvre les deux phases centrales de FORGE : **PLANIFIER** et **IMPLÉMENTER**. Ces phases constituent le cœur de l'hémisphère DELIVERY du SDLC réconcilié. Ce guide est strictement pratique : aucune théorie pure, uniquement des étapes, des gabarits, des exercices et des listes de vérification.

Section ModDevOps	Phases SDLC réconcilié	Guide couleur
SENSE	ÉLICITER · DÉFINIR	Guide Orange
FORGE ← *vous êtes ici*	PLANIFIER · IMPLÉMENTER	**Guide Jaune**
CARE	DIFFUSER · CONFIRMER	Guide Vert

Prérequis pour entrer dans ce guide

- Le **Guide Orange (SENSE)** a été complété ou le backlog contient des spécifications atomiques répondant au DoR.
- L'équipe a identifié les items prêts à entrer en FORGE.
- Au moins un responsable QA (ou son équivalent) participe activement à la planification.

Format du guide

- Chaque module commence par un **objectif clair**.
- Les activités sont guidées étape par étape.
- Les gabarits sont présentés sous forme de **tableaux fillables**.
- Chaque module se termine par une **liste de vérification** ☐.
- Les scénarios DREAM-TEAM illustrent des situations réelles avec tension et résolution.

> **[À retenir]**
>
> Ce guide s'utilise
>
> en tandem avec le Guide Orange (SENSE)
>
> . FORGE commence quand SENSE a produit des spécifications atomiques prêtes — c'est-à-dire quand le DoR (Definition of Ready) est atteint. Sans DoR vérifié, FORGE travaille dans le flou.

Module 1

PLANIFIER : de la spécification au plan de livraison

Objectif : Transformer un backlog affiné en plan de sprint exécutable avec critères de sortie clairs.

1.1 — Comprendre ce que PLANIFIER n'est PAS

Avant de planifier, il est essentiel d'identifier les pièges les plus courants. Voici les cinq erreurs fréquentes observées sur le terrain.

5 erreurs fréquentes en planification

1. **Planifier sans DoR.** Sélectionner des user stories qui n'ont pas encore de critères d'acceptation définis, de tests imaginés, ou d'estimation partagée.
2. **Planifier par habitude de sprint.** Remplir le sprint parce que « c'est l'heure du planning » plutôt que parce que les items sont réellement prêts.
3. **Confondre capacité et vélocité.** Utiliser la vélocité passée comme plafond de planification sans recalculer la capacité réelle du sprint à venir (absences, réunions, congés).
4. **Ignorer les dépendances inter-équipes.** Commencer un item sans confirmer que les livrables amont (APIs, services tiers, données) sont disponibles.
5. **Accepter des spécifications floues dans le sprint.** Valider un item qui dit « améliorer les performances » sans définir ce que « amélioré » signifie en termes mesurables.

> **[Anti-pattern]**
>
> « Le sprint planning qui commence sans que personne n'ait lu les user stories. »
>
> L'équipe arrive au planning. Le PO projette le backlog. Personne ne l'a relu depuis le refinement. Le planning dure 3 heures. Deux stories sont acceptées avec des zones d'ombre. Elles seront renvoyées en milieu de sprint. Le coût : un demi-sprint

> perdu, une frustration collective, une rétrospective difficile.

1.2 — Le DoR : la porte d'entrée de FORGE

> *« Une user story qui entre en FORGE sans DoR est une dette de clarté déguisée en livraison. »*

Le **DoR (Definition of Ready)** est la liste de conditions minimales qu'un item doit satisfaire avant d'être sélectionné dans un sprint. Il n'est pas une bureaucratie — c'est une protection.

Gabarit DoR — Definition of Ready

Critère	Description	Vérifié par	Exemple BEASTEST	☐
Spécification atomique	Format : IF [contexte] THEN [comportement attendu]	Karim (QA)	IF l'utilisateur clique « Exporter PDF » THEN le tableau de bord qualimétrie est généré en ≤ 3 s	☐
Critères d'acceptation définis	≥ 3 exemples concrets couvrant le happy path, une variante et un cas limite	Théo (PO) + Rania	Export PDF valide / Export PDF avec données vides / Export PDF hors délai → message d'erreur	☐
Testabilité	Les critères de	Karim	Scénario	☐

Critère	Description	Vérifié par	Exemple BEASTEST	☐
confirmée (DoT)	test sont rédigés avant l'entrée en sprint		Gherkin rédigé pour les 3 cas d'acceptation	
Dépendances identifiées	Aucune dépendance bloquante non résolue	Rania (tech lead)	API de métriques BEASTEST disponible en staging	☐
Estimation partagée	L'équipe a estimé ensemble (planning poker ou technique d'équipe)	DREAM-TEAM	5 points — consensus atteint au 2e tour	☐
Priorité validée	Business value connue et confirmée par le PO pour ce sprint	Théo	Priorité haute — demande de la direction qualité d'ORIGINALIUM	☐
Impact UX/UI clarifié	Les changements visuels ou d'interface sont documentés ou maquettés	Rania + Théo	Maquette Figma validée par Théo le sprint précédent	☐
Critère de fin identifiable	L'équipe peut dire « c'est fini » de manière non ambiguë	DREAM-TEAM	« Le PDF est généré, téléchargeable et non corrompu »	☐

Usage du DoR

- ☐ Appliquer le DoR lors du refinement (avant le sprint planning)
- ☐ Refuser tout item dont ≥ 2 critères ne sont pas satisfaits

- ☐ Revérifier le DoR en début de sprint planning avant la sélection finale
- ☐ Mettre à jour le DoR après chaque rétrospective si un critère manque

---

1.3 — Le sprint planning ToTAL : 4 étapes

Le sprint planning dans FORGE n'est pas une réunion de négociation. C'est une séance de vérification et d'engagement.

Étape	Activité	Durée	Participants	Livrable
1	Lecture partagée des spécifications (Three Amigos en format concentré)	15 min	Rania + Théo + Karim	Chaque item relu par les 3 perspectives : tech / produit / qualité
2	Vérification DoR pour chaque item sélectionné	10 min / item	DREAM-TEAM	Checklist DoR ☐ complétée — items non prêts retirés du sprint
3	Décomposition en tâches techniques	20 min / item	Rania (lead) + équipe	Tableau de tâches avec effort estimé et responsable
4	Engagement de sprint = capacité réelle, pas optimiste	15 min	DREAM-TEAM	Backlog de sprint engagé, objectif de sprint rédigé

Gabarit : Canevas de Sprint Planning

Item backlog	DoR ✓/✗	Décomposition tâches	Effort estimé (pts)	Assigné	Signal de fin (DoD partiel)
Dashboard qualimétrie	✓	1. Intégration	5	Rania	PDF généré,

Item backlog	DoR ✓/✗	Décomposition tâches	Effort estimé (pts)	Assigné	Signal de fin (DoD partiel)
— Export PDF		lib PDF · 2. Template HTML → PDF · 3. Test export · 4. Revue Karim			pipeline vert, revue faite
Filtres par module BEASTEST	✓	1. API filtres · 2. UI filtre · 3. Tests unitaires · 4. Revue Karim	3	Rania + Théo	Filtres fonctionnels en staging, demo validée
Alertes seuil qualité	✗ — DoT manquant	— reporté au sprint suivant —	—	—	—
[Item suivant]	☐				

[Scénario DREAM-TEAM — Sprint 14, ORIGINALIUM]

Rania, Théo et Karim démarrent le sprint planning du Sprint 14 de BEASTEST. Théo présente 9 items. Karim vérifie le DoR item par item. Trois items sont retenus en moins d'une heure — dont l'item « Alertes seuil qualité » est

retiré

car les critères de testabilité (DoT) ne sont pas rédigés. Théo est frustré. Rania rappelle la règle : « Un item non testable n'est pas un item livrable. » La réunion se termine en 90 minutes. Tout le monde sait exactement ce qui entre dans le sprint.

1.4 — La capacité réelle vs la capacité fantasmée

La majorité des sprints dérapent non pas par manque de compétence, mais par surestimation de la capacité disponible.

Formule ToTAL — Capacité réelle

> Capacité réelle = (Jours ouvrables × Membres disponibles) × Facteur d'absorption (0,6 à 0,8)

Le **facteur d'absorption** (0,6–0,8) tient compte des réunions, interruptions, onboarding, révisions et imprévus. Il n'est jamais 1,0 sur un sprint normal.

Membre	Jours disponibles (sprint 2 sem.)	Absence / réunions hors sprint	Jours effectifs	Facteur absorption	Capacité réelle (jours)
Rania (tech lead)	10	1,5 (réunions archi)	8,5	0,75	**6,4**
Théo (PO)	10	3 (stakeholders)	7	0,70	**4,9**
Karim (QA)	10	1 (formation)	9	0,80	**7,2**
Total équipe	30	5,5	24,5	—	**18,5 jours-équipe**

> **[Terrain]**
>
> Au Sprint 12 de BEASTEST, Karim propose de réserver
>
> 20 % du sprint en marge de qualité

> — soit environ 3,7 jours-équipe dédiés aux tests de régression, aux revues de code et à la stabilisation. Rania et Théo résistent dans un premier temps. Après une rétrospective difficile où deux bugs critiques avaient glissé jusqu'en staging, ils acceptent. Le Sprint 13 est le premier sans bug bloquant en revue de sprint.

1.5 — Les dépendances : cartographier avant de commencer

Une dépendance non identifiée en planification devient un bloqueur en cours de sprint. La carte des dépendances prend 20 minutes à construire et évite des jours de blocage.

Gabarit : Carte des dépendances de sprint

Item backlog	Dépend de	Responsable de la dépendance	Statut à J-0 sprint	Risque si bloqué	☐
Export PDF dashboard	API métriques BEASTEST v2.1	Équipe backend ORIGINALIUM	Disponible en staging	Faible	☐
Filtres par module	Schéma base données v3	Rania	Migration planifiée J+2	Moyen — démarrer UI d'abord	☐
Alertes seuil qualité	DoT rédigé par Karim	Karim	Non disponible — item retiré	Item absent du sprint	☐
[Item]					☐
[Item]					☐
[Item]					☐

Liste de vérification — Dépendances ☐ ×6

- ☐ Toutes les dépendances de chaque item ont été listées explicitement
- ☐ Chaque dépendance a un responsable nommé
- ☐ Le statut de chaque dépendance est confirmé (pas présumé)
- ☐ Les dépendances à risque moyen/élevé ont un plan de contournement
- ☐ Les items avec dépendances critiques non résolues ont été retirés ou replanifiés
- ☐ La carte des dépendances est accessible à toute l'équipe pendant le sprint

---

1.6 — Checklist de fin — MODULE 1 : PLANIFIER

☐ **Checklist MODULE 1 — Avant de passer à IMPLÉMENTER**

- ☐ DoR vérifié pour tous les items sélectionnés dans le sprint (aucune exception)
- ☐ Capacité réelle calculée avec facteur d'absorption appliqué (pas la vélocité brute)
- ☐ Dépendances cartographiées — aucune dépendance bloquante non résolue
- ☐ Critères d'acceptation relus par Karim (QA) lors du planning
- ☐ Gabarit de sprint planning complété (items, tâches, efforts, assignations)
- ☐ Équipe alignée sur l'objectif du sprint (rédigé en une phrase)
- ☐ Marge de qualité réservée dans la capacité du sprint (recommandé : 20

%)

- ☐ Items rejetés documentés avec la raison (critère DoR non satisfait)

Module 2

IMPLÉMENTER : construire avec discipline de qualité intégrée

Objectif : Livrer chaque item du sprint avec preuve de qualité intégrée — pas ajoutée en fin de cycle.

2.1 — Ce que IMPLÉMENTER signifie dans ToTAL

Dans la plupart des approches, IMPLÉMENTER signifie « coder ». Dans FORGE, IMPLÉMENTER signifie **construire + tester + documenter + intégrer** — et ces quatre activités se déroulent **en parallèle**, pas en séquence.

Dimension	Approche traditionnelle	FORGE (ToTAL)
Test	Phase après le code	Écrit avant ou en parallèle du code (DoT)
Documentation	Rédigée en fin de sprint	Mise à jour à chaque merge
Intégration	Intégration de fin de sprint (« big bang »)	Continue — pipeline CI vert obligatoire
Qualité	Vérifiée par QA en fin de cycle	Condition du code — Karim dans la boucle dès J1

[À retenir]

> Dans FORGE, la qualité n'est pas une phase après le code. Elle est la
>
> condition du code
>
> . Un item sans tests n'est pas un item terminé — c'est un item à risque déguisé en
>
> livraison.

2.2 — Le flux de travail IMPLÉMENTER : 6 micro-étapes

Chaque item du sprint suit ce flux. Toute déviation est un signal à nommer en Daily FORGE.

Étape	Activité	Responsable	Condition de passage	Signal d'alerte
1	Prendre l'item — vérifier que le DoR est toujours satisfait	Développeur assigné + Karim	DoR ☐ encore valide	DoR devenu invalide depuis le planning → bloquer et notifier
2	Écrire le test avant ou en parallèle du code (critères DoT)	Développeur + Karim	Au moins un test par critère d'acceptation rédigé	Code démarré sans test → signaler en Daily FORGE
3	Coder la solution minimale répondant aux critères d'acceptation	Développeur assigné	Tous les cas d'acceptation couverts par le code	Over-engineering détecté → revue anticipée
4	Intégration continue — pipeline CI vert obligatoire avant push	Développeur + CI/CD	Pipeline 100 % vert	Pipeline rouge → merge bloqué, correction prioritaire
5	Revue de code (Rania + Karim) —	Rania (tech lead) + Karim (QA)	Revue complétée, commentaires	Revue sautée → item non éligible au

Étape	Activité	Responsable	Condition de passage	Signal d'alerte
	gabarit checklist revue		adressés	DoD
6	Vérification DoD avant de marquer Done	DREAM-TEAM	Tous les critères DoD satisfaits ☐	DoD partiel → item reste In Progress

---

2.3 — Le DoD : la porte de sortie de FORGE

Le **DoD (Definition of Done)** définit ce que signifie « terminé » de manière non ambiguë. Un item est Done quand chaque critère est satisfait — pas quand le code est écrit.

Gabarit DoD — Definition of Done

Critère	Description	Responsable	Exemple BEASTEST	☐
Code intégré	Merge dans la branche principale sans conflit résolu à la hâte	Rania	PR mergée dans main, aucun conflit non résolu	☐
Pipeline CI vert	Tous les tests automatisés passent dans la branche principale	CI/CD (auto)	GitHub Actions : 100 % green, 0 warning bloquant	☐
Tests unitaires écrits	Couverture ≥ seuil défini par l'équipe sur la fonctionnalité	Karim	Couverture ≥ 85 % sur le module PDF export	☐
Critères d'acceptation validés	Tests de comportement (BDD) exécutés et tous verts	Karim + Théo	Scénarios Gherkin exécutés : 3/3 passent	☐
Revue de	≥ 1 pair	Rania	Revue Rania	☐

Critère	Description	Responsable	Exemple BEASTEST	☐
code faite	reviewer a approuvé — commentaires adressés		+ 1 commentaire Karim adressé	
Documentation à jour	Changelog mis à jour, commentaires critiques dans le code	DREAM-TEAM	CHANGELOG.md : entrée v2.4.0 rédigée	☐
Démo possible	Feature démontrée en environnement d'intégration (pas local)	Rania + Théo	Export PDF démontré en staging devant Théo	☐
Régressions vérifiées	Suite de non-régression exécutée sans nouvelles défaillances	Karim	Suite 147 tests — 0 régression sur fonctions existantes	☐

> **[Signal]**
>
> Le DoD n'est pas un formulaire à cocher après coup. Il se prépare pendant
>
> PLANIFIER et se valide
>
> avant de déclarer Done
>
> . Si un critère manque, l'item reste « In Progress » — même si le code est mergé.

2.4 — Gabarit : Revue de code ToTAL

La revue de code dans FORGE n'est pas une vérification de style. C'est une validation de qualité intégrée.

Dimension	Questions clé de la revue	Commentaire si non	☐
Clarté	Le code exprime-t-il l'intention clairement sans commentaire requis ?	Renommer variables / extraire méthode	☐
Complétude	Tous les critères DoD sont-ils couverts par cette implémentation ?	Lister les critères manquants	☐
Testabilité	Les tests couvrent-ils le happy path, les variantes et les cas limites ?	Ajouter tests pour cas limites identifiés	☐
Cohérence architecturale	Le code s'intègre-t-il avec l'architecture existante de BEASTEST ?	Escalader à Rania pour décision archi	☐
Signal qualité	Y a-t-il des dettes techniques à nommer explicitement ?	Documenter dans le registre de dette technique	☐
Sécurité	Y a-t-il des risques de sécurité (injection, exposition de données) ?	Bloquer le merge — escalader immédiatement	☐
Performance	Y a-t-il des opérations non scalables introduites dans ce code ?	Documenter le contexte et le seuil acceptable	☐

2.5 — Scénario DREAM-TEAM : Semaine 3 de Sprint 14

[Scénario DREAM-TEAM — Sprint 14, Semaine 3, ORIGINALIUM]

Jeudi matin. La feature de dashboard qualimétrie passe tous les tests unitaires

en local et en CI. Karim lance la suite d'intégration complète en staging. Résultat : **la feature passe, mais trois métriques affichent 0 sans message d'erreur** — un échec silencieux. Le pipeline est vert. Le bug est invisible pour un œil non averti.

Karim alerte Rania à 10h17. Ils organisent une **micro-rétrospective de 20 minutes** en après-midi avec Théo. Diagnostic : le DoD ne mentionne pas les tests d'intégration explicitement — seulement les tests unitaires et le pipeline CI. La métrique vide est un cas limite non couvert par les critères d'acceptation originaux.

Décision collective : Ajout d'un critère DoD : « Tests d'intégration en staging exécutés et aucune métrique silencieuse. » BEASTY, l'IA intégrée dans BEASTEST, suggère un pattern de *health-check monitoring* sur les endpoints de métriques pour détecter ce type d'échec automatiquement. Rania l'ajoute au registre de dette technique — à implémenter en Sprint 15.

Le sprint continue. La feature est corrigée en 4 heures. Le nouveau critère DoD est en place pour tous les sprints suivants.

2.6 — La dette technique : nommer sans dramatiser

La dette technique n'est pas un échec. C'est une **décision consciente** de livrer maintenant et d'adresser plus tard — à condition d'être nommée, documentée et révisée.

> **[Signal]**
>
> « Une dette technique nommée est une décision. Une dette technique ignorée est un risque. »

Gabarit : Registre de dette technique FORGE

Item de dette	Décision consciente (pourquoi maintenant ?)	Impact estimé si non adressée	Responsable	Date de revue prévue	☐ Adressée
Health-check monitoring endpoints métriques	Priorisé livraison dashboard Sprint 14 — pattern suggéré par BEASTY	Risque d'échec silencieux sur nouveaux endpoints	Rania	Sprint 15, J1	☐
Optimisation requête filtres module	N+1 queries acceptées provisoirement — volume actuel tolérable	Dégradation performance si +500 modules BEASTEST	Rania	Sprint 16	☐
[Item de dette]					☐
[Item de dette]					☐

2.7 — Checklist de fin — MODULE 2 : IMPLÉMENTER

☐ **Checklist MODULE 2 — Avant de passer au Go/No-Go**

- ☐ DoT respecté — tests écrits avant ou en parallèle du code pour chaque item
- ☐ Pipeline CI vert avant chaque merge (aucun merge sur pipeline rouge)
- ☐ DoD vérifié item par item — aucun item marqué Done avec critère manquant
- ☐ Revue de code complétée par ≥ 1 pair reviewer pour chaque item

- ☐ Dettes techniques identifiées nommées dans le registre (pas ignorées)
- ☐ Démo possible en environnement d'intégration (pas en local)
- ☐ Suite de non-régression exécutée sans nouvelles défaillances
- ☐ Documentation (changelog, commentaires) mise à jour pour chaque item livré

Module 3

Le rythme FORGE : cérémonies et signaux

Objectif : Maintenir le rythme de livraison sans surcharger l'équipe de réunions improductives.

3.1 — Les 4 cérémonies FORGE

FORGE fonctionne avec quatre cérémonies. Chacune a une durée maximale, un livrable attendu et une règle d'or.

Cérémonie	Fréquence	Durée max.	Participants	Livrable concret	Règle d'or
Sprint Planning	Début de sprint	2 h	DREAM-TEAM complète	Backlog de sprint engagé, objectif rédigé	Pas d'item sans DoR ✓
Daily FORGE	Quotidien	15 min	DREAM-TEAM	≤ 3 blockers identifiés et assignés	3 questions uniquement — pas de rapport de statut
Revue de sprint	Fin de sprint	1 h	DREAM-TEAM +	Demo réalisée +	Demo en environnem

Cérémonie	Fréquence	Durée max.	Participants	Livrable concret	Règle d'or
			stakeholders	feedback collecté	ent d'intégration, jamais en local
Rétrospective FORGE	Fin de sprint	1 h	DREAM-TEAM	≥ 1 amélioration DoD ou DoR appliquée	L'action d'amélioration est ajoutée au backlog — pas ignorée

---

3.2 — Le Daily FORGE : format ToTAL (3 questions)

Le Daily FORGE n'est pas un rapport de statut. C'est un mécanisme de détection précoce des blockers et des signaux qualité.

#	Question ToTAL	Intention — ce qu'on cherche à détecter
1	Qu'ai-je livré hier qui répond au DoD ?	Vérifier que le travail accompli est réellement Done — pas juste codé
2	Qu'est-ce qui bloque mon chemin vers le DoD aujourd'hui ?	Nommer les blockers avant qu'ils durent 24h — technique, dépendance, ambiguïté
3	Y a-t-il un signal de qualité à nommer à l'équipe ?	Partager une observation de qualité (test fragile, dette détectée, régression possible)

> **[Anti-pattern]**
>
> Le Daily devenu réunion de rapport :
>
> « Hier j'ai travaillé sur la feature X. Aujourd'hui je continue. Pas de bloquant. » Ce format ne détecte rien. Le Daily FORGE cherche des signaux — pas des résumés.

> Si la réponse à la question 3 est toujours « non », l'équipe ne regarde pas assez près.

3.3 — La rétrospective FORGE : améliorer DoR et DoD

La rétrospective FORGE a une règle absolue : **elle doit produire au moins une modification du DoR ou du DoD**. Sinon, elle n'a servi qu'à constater — pas à améliorer.

Gabarit de rétrospective FORGE

Ce qui a bien fonctionné *(conserver et amplifier)*	**Ce qui a bloqué *(nommer sans jugement)***	**Ce qu'on améliore *(action concrète + responsable)***
Exemple : Pipeline CI stable — 0 merge rouge	Exemple : Tests d'intégration absents du DoD — bug silencieux non détecté	Exemple : Ajouter « Tests d'intégration staging » au DoD — Karim applique ce sprint

Action obligatoire de rétrospective :

- ☐ Au moins **un critère DoR ou DoD** est modifié, ajouté ou clarifié suite à la rétrospective.
- ☐ L'action d'amélioration est ajoutée au **backlog d'équipe** avec un responsable nommé.
- ☐ L'amélioration est vérifiée lors de la **rétrospective suivante**.

3.4 — Les signaux de santé d'un sprint FORGE

Dimension	Signal vert ✓	Signal orange — attention	Signal rouge — agir maintenant
Pipeline CI	**Vert ≥ 90 % du temps**	**Instable — rouge récurrent <24h**	**Rouge >24h consécutives — arrêt livraison**
DoD atteint	**≥ 80 % des items engagés terminent Done**	**DoD partiel sur >30 % des items**	**Items reportés >50 % — sprint non livré**
Rétrospective	**Productive — ≥ 1 amélioration DoR/DoD appliquée**	**Superficielle — aucun changement DoR/DoD**	**Rétrospective sautée 2 sprints consécutifs**
Dette technique	**Nommée et documentée dans le registre**	**Ignorée en cours de sprint**	**Croissante silencieusement — registre vide**
Daily FORGE	**3 questions répondues — blockers résolus <24h**	**Blockers non résolus >48h**	**Daily sautée ou devenue rapport statut**
Revue de sprint	**Demo réalisée en staging — feedback collecté**	**Demo partielle ou en environnement local**	**Revue sans demo — sprint non démontrable**
DoR en planning	**100 % des items sélectionnés ont DoR ✓**	**1 item accepté avec DoR partiel**	**Planification sans DoR — règle ignorée**

☐ Checklist MODULE 3 — Rythme FORGE en place

- ☐ Sprint Planning < 2h avec DoR vérifié pour tous les items sélectionnés
- ☐ Daily FORGE quotidien — 3 questions, ≤ 15 min
- ☐ Revue de sprint réalisée en staging avec stakeholders présents
- ☐ Rétrospective FORGE tenue — ≥ 1 amélioration DoR/DoD actée
- ☐ Tableau de signaux de santé relu en rétrospective

- ☐ Blockers adressés dans les 24h suivant leur identification

---

Module 4

Le Go/No-Go de FORGE

Objectif : Décider avec lucidité si la livraison est prête pour CARE (DIFFUSER) — sans pression de calendrier artificielle.

4.1 — Le Go/No-Go : pas une formalité, une décision

Le Go/No-Go marque la frontière entre FORGE et CARE dans le SDLC réconcilié. C'est le moment où l'équipe, collectivement, décide : cette livraison est-elle prête à passer en production ?

> **[À retenir]**
>
> Un Go/No-Go honnête protège la confiance accumulée sur tout le sprint. Un Go forcé par la pression du calendrier détruit ce que FORGE a construit — et transfère le risque vers les utilisateurs finaux de BEASTEST.

Le Go/No-Go n'est pas un vote. C'est une **lecture des preuves**. Si les preuves sont là, le Go s'impose. Si elles manquent, le No-Go s'impose — indépendamment de la date de livraison prévue.

4.2 — Gabarit Go/No-Go de fin de FORGE

Dimension	Critère	Statut	Preuve (lien, screenshot, rapport)
Qualité technique	DoD atteint sur 100 % des items engagés dans le sprint	☐ Go ☐ No-Go	
Pipeline CI/CD	Pipeline vert en environnement staging — 0 test en échec	☐ Go ☐ No-Go	
Tests de régression	Suite non-régression exécutée sans nouvelles défaillances	☐ Go ☐ No-Go	
Documentation	Release notes rédigées et approuvées	☐ Go ☐ No-Go	
Demo PO	Demo acceptée par Théo (PO) en staging	☐ Go ☐ No-Go	
Risques résiduels	Risques résiduels nommés, documentés et acceptés consciemment	☐ Go ☐ No-Go	
Plan de rollback	Procédure de rollback définie et testée (ou au minimum documentée)	☐ Go ☐ No-Go	

Règle de décision :

- Si toutes les dimensions sont **Go** → passer à CARE (DIFFUSER).
- Si ≥ 1 dimension est **No-Go** → corriger, ne pas passer à CARE. Documenter la raison.
- Si une dimension est No-Go mais acceptée consciemment (risque résiduel nommé) → décision collective documentée, responsable nommé.

4.3 — Le No-Go sans honte : scénario DREAM-TEAM

[Scénario DREAM-TEAM — Sprint 14, Vendredi 16h, ORIGINALIUM]

Le Sprint 14 s'achève. La revue de sprint a été positive. Théo a validé la demo. Les équipes sont prêtes pour le Go/No-Go.

À 16h02, Karim lance la suite de non-régression finale. À 16h19, un test échoue. Une régression sur la fonctionnalité d'export CSV — fonctionnalité existante, non touchée dans ce sprint, mais impactée par la migration du schéma base de données planifiée en milieu de sprint.

Karim alerte Rania. Rania consulte Théo. La conversation dure **10 minutes**.

Rania : « C'est un No-Go. »
Théo : « Je confirme. On ne sort pas avec une régression sur l'export CSV — c'est critique pour les utilisateurs actuels de BEASTEST. »
Karim : « Je commence l'analyse ce soir. Lundi matin on a le correctif. »

La décision est prise sans honte, sans débat prolongé, sans pression. La semaine suivante, le correctif est propre. Le Sprint 15 démarre avec un Go solide. La confiance de l'équipe et des stakeholders reste intacte — précisément parce que le No-Go a été honoré.

4.4 — Checklist Go/No-Go

☐ **Checklist MODULE 4 — Go/No-Go FORGE**

- ☐ DoD vérifié à 100 % sur tous les items engagés (pas 80 %, pas « presque »)
- ☐ Pipeline CI/CD vert en staging au moment de la décision

- ☐ Suite de non-régression exécutée — résultats documentés
- ☐ Release notes rédigées et accessibles
- ☐ Demo validée par Théo (PO) en environnement d'intégration
- ☐ Risques résiduels nommés et acceptés collectivement par la DREAM-TEAM
- ☐ Plan de rollback défini — responsable nommé si activation requise

---

Module 5

FORGE et l'IA : BEASTY dans la boucle

Objectif : Utiliser l'IA comme amplificateur de discipline — pas comme raccourci vers le flou ou substitut au jugement humain.

5.1 — Ce que BEASTY fait dans FORGE

BEASTY est l'assistant IA intégré dans BEASTEST chez ORIGINALIUM. Dans FORGE, BEASTY agit comme un **amplificateur de discipline** : il augmente la capacité de l'équipe à détecter les écarts sans remplacer le jugement humain.

Capacité BEASTY	Description	Exemple dans BEASTEST
Décomposition de user stories	Suggère des tâches techniques à partir des critères d'acceptation	À partir des 3 critères d'acceptation export PDF, BEASTY propose 6 tâches techniques ordonnées
Analyse couverture DoD	Compare les commits du sprint aux critères DoD — identifie les gaps potentiels	BEASTY détecte qu'aucun commit ne touche le module de documentation — DoD « changelog »

Capacité BEASTY	Description	Exemple dans BEASTEST
		potentiellement non couvert
Détection de patterns de dette technique	Identifie des patterns connus (N+1, couplage fort, absence de gestion d'erreur) dans le code soumis en revue	BEASTY signale 2 endpoints sans gestion d'exception — Rania les traite lors de la revue
Génération de release notes	Produit un brouillon de release notes à partir des commits et des critères DoD satisfaits	Brouillon CHANGELOG v2.4.0 généré en 2 minutes — Théo révise et valide
Suggestions de monitoring	Propose des patterns de surveillance pour les fonctionnalités livrées	Pattern health-check endpoints métriques — suggéré après l'incident Sprint 14

5.2 — Ce que BEASTY NE remplace PAS

Les limites de BEASTY dans FORGE ne sont pas des limitations techniques — ce sont des **frontières intentionnelles**. Certaines décisions doivent rester humaines.

Ce que BEASTY ne remplace pas	Pourquoi cette frontière est intentionnelle
Le jugement Go/No-Go	La décision de livrer engage la responsabilité de l'équipe envers les utilisateurs de BEASTEST. Une IA peut signaler des risques — elle ne peut pas assumer cette responsabilité.
La revue de code entre pairs	La revue humaine détecte l'intention derrière le code, les décisions d'architecture implicites, et les dettes contextuelles qu'un outil ne connaît pas.
La rétrospective d'équipe	La rétrospective crée de la confiance, de la sécurité psychologique et des engagements collectifs. Ces dynamiques sont humaines par nature.
La décision de priorisation (Théo reste PO)	La priorisation du backlog reflète des arbitrages de valeur métier, de

Ce que BEASTY ne remplace pas	Pourquoi cette frontière est intentionnelle
	confiance client et de stratégie produit. Théo détient ce contexte — BEASTY ne le connaît pas.
La définition des critères DoR et DoD	DoR et DoD sont des accords d'équipe — ils reflètent la maturité, le contexte et les engagements collectifs. Ils sont définis PAR l'équipe, validés en rétrospective.

5.3 — Exercice guidé : BEASTY analyse le sprint

> *Cet exercice entraîne l'équipe à utiliser BEASTY comme outil de vérification — et à évaluer la pertinence de ses suggestions par rapport au jugement humain.*

Contexte

Rania soumet les 12 commits du Sprint 14 de BEASTEST à BEASTY en fin de sprint.

Question soumise à BEASTY

> *« Quels critères DoD semblent non couverts ou partiellement couverts dans les 12 commits de ce sprint ? »*

Livrable BEASTY (exemple de réponse attendue)

Critère DoD	Statut selon BEASTY	Recommandation BEASTY
Tests unitaires écrits	Couvert — 14 nouveaux fichiers test détectés	Vérifier couverture sur module PDF

Critère DoD	Statut selon BEASTY	Recommandation BEASTY
		spécifiquement
Documentation à jour	Potentiellement non couvert — aucun commit CHANGELOG.md	Rédiger l'entrée changelog avant le Go/No-Go
Tests d'intégration staging	Non détectable via commits — vérification manuelle requise	Confirmer avec Karim que la suite staging a tourné
Revue de code	10/12 PRs avec approbation détectée	2 PRs sans approbation — vérifier si revue réalisée hors plateforme

Exercice pratique

6. Comparer la liste BEASTY avec la **checklist DoD manuelle** complétée par Karim.
7. Identifier : **Où concordent-ils ?** (BEASTY et jugement humain alignés)
8. Identifier : **Où divergent-ils ?** (BEASTY a détecté quelque chose que l'humain a manqué — ou vice versa)
9. Documenter les divergences dans un tableau de retour d'expérience.
10. Décider collectivement : faut-il ajuster le DoD ou les pratiques de commit pour rendre la vérification BEASTY plus fiable ?

Critère DoD	Évaluation BEASTY	Évaluation Karim (humain)	Concordance ✓/✗	Action si divergence

---

5.4 — Garde-fous IA dans FORGE

☐ **Garde-fous BEASTY — Protocole d'usage dans FORGE**

- ☐ Toute suggestion de BEASTY est **validée par l'équipe** avant toute action — jamais appliquée automatiquement
- ☐ BEASTY ne remplace **jamais** la revue de code humaine — il la complète
- ☐ Les critères DoR et DoD sont définis **par l'équipe** en rétrospective — jamais générés par BEASTY
- ☐ Les décisions Go/No-Go restent **humaines et collectives** — BEASTY peut signaler des risques, pas décider
- ☐ Les sorties BEASTY sont traitées comme des **suggestions à valider**, pas comme des vérités
- ☐ L'équipe documente **chaque divergence** entre l'analyse BEASTY et le jugement humain pour améliorer les pratiques

> **[À retenir]**
>
> BEASTY amplifie ce que l'équipe sait déjà faire. Si l'équipe n'a pas de DoD solide, BEASTY ne peut pas compenser cette absence. L'IA dans FORGE est aussi bonne que la discipline humaine qu'elle soutient.

Annexe J1 — Lexique FORGE

Terme	Définition FORGE (Guide Jaune)
PLANIFIER	Phase de FORGE qui transforme un backlog affiné en plan de sprint exécutable. PLANIFIER commence quand le DoR est atteint et se termine quand le sprint est engagé avec capacité réelle calculée et dépendances cartographiées.
IMPLÉMENTER	Phase de FORGE qui produit la livraison réelle : code + tests + documentation + intégration, en parallèle, avec le DoD comme critère de sortie obligatoire. IMPLÉMENTER n'est pas seulement coder — c'est livrer avec preuve de qualité intégrée.
DoR (Definition of Ready)	Liste de conditions qu'un item doit satisfaire avant d'entrer en sprint. Le DoR protège l'équipe contre la planification de travail ambigu. Dans FORGE, un item sans DoR est retourné au backlog — pas accepté avec des hypothèses.
DoD (Definition of Done)	Liste de conditions qu'un item doit satisfaire pour être déclaré terminé. Le DoD est l'accord d'équipe sur ce que signifie « fini » de manière non ambiguë. Il évolue à chaque rétrospective et s'applique à tous les items sans exception.
DoT (Definition of Testable)	Critères de testabilité écrits pour un item avant qu'il entre en sprint. Le DoT garantit que Karim (QA) peut écrire ou valider des tests dès le début de l'implémentation. Un item sans DoT n'est pas prêt (critère DoR).
Go/No-Go	Décision collective et documentée de passer (Go) ou de ne pas passer (No-Go) de FORGE à CARE (DIFFUSER). Le Go/No-Go repose sur des preuves concrètes — pas sur la pression du calendrier. Un No-Go honnête protège la confiance.
Sprint	Itération de durée fixe (généralement 2 semaines) pendant laquelle l'équipe planifie, implémente et démontre un ensemble d'items engagés. Dans FORGE, un sprint est un cycle complet de PLANIFIER + IMPLÉMENTER.

Terme	Définition FORGE (Guide Jaune)
Pipeline CI/CD	Ensemble automatisé de vérifications (tests unitaires, intégration, déploiement) déclenchées à chaque commit ou merge. Dans FORGE, le pipeline doit être vert avant tout merge — un pipeline rouge bloque la livraison.
Vélocité	Mesure historique du nombre de points livrés par sprint. La vélocité est un indicateur passé — pas une promesse de capacité future. Dans FORGE, elle sert de référence, pas de cible imposée.
Capacité réelle	Nombre de jours-équipe réellement disponibles dans un sprint, calculé avec le facteur d'absorption (0,6–0,8). La capacité réelle est toujours inférieure à la capacité nominale. La confondre avec la vélocité est une erreur fréquente de planification.
Dette technique	Décision consciente de livrer une solution imparfaite maintenant et de l'améliorer plus tard. Dans FORGE, toute dette technique est nommée dans le registre de dette — avec son impact estimé et sa date de revue. Une dette ignorée est un risque non géré.
Non-régression	Suite de tests automatisés vérifiant que les fonctionnalités existantes n'ont pas été cassées par les nouvelles livraisons. Dans FORGE, la non-régression est exécutée avant le Go/No-Go — pas optionnelle.

Annexe J2 — Gabarits récapitulatifs

Gabarit DoR — Récapitulatif (8 critères)

#	Critère DoR	Vérifié par	☐
1	Spécification	Karim (QA)	☐

#	Critère DoR	Vérifié par	☐
	atomique (IF/THEN)		
2	Critères d'acceptation définis (≥ 3 exemples)	Théo (PO) + Rania	☐
3	Testabilité confirmée (DoT rédigé)	Karim	☐
4	Dépendances identifiées — aucune bloquante non résolue	Rania (tech lead)	☐
5	Estimation partagée par l'équipe	DREAM-TEAM	☐
6	Priorité validée par le PO (business value connue)	Théo	☐
7	Impact UX/UI clarifié (maquette ou description)	Rania + Théo	☐
8	Critère de fin identifiable sans ambiguïté	DREAM-TEAM	☐

Gabarit DoD — Récapitulatif (8 critères)

#	Critère DoD	Responsable	☐
1	Code intégré — merge dans branche principale sans conflit	Rania	☐
2	Pipeline CI vert — tous les tests automatisés passent	CI/CD (auto)	☐
3	Tests unitaires écrits — couverture ≥ seuil défini	Karim	☐

#	Critère DoD	Responsable	☐
4	Critères d'acceptation validés — scénarios BDD exécutés	Karim + Théo	☐
5	Revue de code faite — ≥ 1 pair reviewer	Rania	☐
6	Documentation à jour — changelog + commentaires critiques	DREAM-TEAM	☐
7	Démo possible en environnement d'intégration (staging)	Rania + Théo	☐
8	Régressions vérifiées — suite non-régression exécutée	Karim	☐

Gabarit Go/No-Go — Récapitulatif (7 dimensions)

#	Dimension	Critère	Statut	Preuve
1	Qualité technique	DoD 100 % items engagés	☐ Go ☐ No-Go	
2	Pipeline CI/CD	Vert en staging	☐ Go ☐ No-Go	
3	Non-régression	Suite exécutée — 0 nouvelle défaillance	☐ Go ☐ No-Go	
4	Documentation	Release notes rédigées	☐ Go ☐ No-Go	
5	Demo PO	Acceptée par Théo en staging	☐ Go ☐ No-Go	
6	Risques résiduels	Nommés et acceptés consciemment	☐ Go ☐ No-Go	

#	Dimension	Critère	Statut	Preuve
7	Rollback	Plan défini et responsable nommé	☐ Go ☐ No-Go	

Gabarit Daily FORGE (3 questions)

#	Question ToTAL	Réponse du jour
1	Qu'ai-je livré hier qui répond au DoD ?	
2	Qu'est-ce qui bloque mon chemin vers le DoD aujourd'hui ?	
3	Y a-t-il un signal de qualité à nommer à l'équipe ?	

Gabarit Rétrospective FORGE

Sprint #	Ce qui a bien fonctionné	Ce qui a bloqué	Ce qu'on améliore (action + responsable + date)

Action DoR/DoD : *Critère modifié/ajouté : _____*
Responsable : _____ Appliqué dès : ____

Annexe J3 — Signaux de qualité FORGE : tableau de bord (15 signaux)

#	Dimension	Signal vert	Signal orange	Signal rouge
1	Pipeline CI	**Vert ≥ 90 % du temps**	**Rouge récurrent <24h**	**Rouge >24h — merge bloqué**
2	DoD — taux de complétion	**≥ 80 % items Done en fin de sprint**	**DoD partiel >30 % items**	**Items reportés >50 %**
3	DoR — taux de conformité planning	**100 % items avec DoR ✓ en planning**	**1 item accepté sans DoR complet**	**Planning sans DoR — règle ignorée**
4	Couverture de tests	**Couverture ≥ seuil défini sur toutes les features**	**Couverture en baisse sur 1 module**	**Feature livrée sans tests**
5	Non-régression	**0 régression sur 3 sprints consécutifs**	**1 régression détectée en staging**	**Régression en production**
6	Revues de code	**100 % PRs avec ≥ 1 reviewer approuvé**	**PRs mergées sans revue formelle**	**Revue de code absente du process**
7	Dette technique	**Registre à jour — dettes nommées et planifiées**	**Dettes détectées non enregistrées**	**Registre vide — dettes croissantes**
8	Daily FORGE	**3 questions répondues, ≤ 15 min**	**Daily >20 min ou questions ignorées**	**Daily non tenu >2 jours**
9	Blockers	**Blocker résolu <24h après identification**	**Blocker non résolu 24–48h**	**Blocker >48h sans escalade**
10	Rétrospective	**Tenue — ≥ 1 amélioration DoR/DoD actée**	**Tenue — aucun changement DoR/DoD**	**Sautée ≥ 2 sprints consécutifs**
11	Démo de sprint	**Demo en staging — feedback**	**Demo en local ou partielle**	**Pas de demo — sprint non visible**

#	Dimension	Signal vert	Signal orange	Signal rouge
		collecté		
12	Capacité planning	**Capacité réelle calculée avec facteur absorption**	**Capacité estimée sans facteur absorption**	**Vélocité brute = capacité planifiée**
13	Go/No-Go	**Décision documentée avec preuves**	**Décision prise informellement**	**Go forcé sans checklist — risque ignoré**
14	Documentation	**Changelog mis à jour à chaque livraison**	**Changelog en retard de 1 sprint**	**Changelog absent — livraisons non traçables**
15	Usage BEASTY (IA)	**Suggestions validées par l'équipe avant action**	**Suggestions appliquées sans validation**	**BEASTY remplace une décision humaine critique**

Usage du tableau de bord : *Relire en rétrospective à chaque fin de sprint. Tout signal orange déclenche une discussion. Tout signal rouge déclenche une action dans le backlog d'équipe. Les 15 signaux sont évalués collectivement — pas par un seul membre.*

Auteur : Léo Michel Bélanger | Tous droits réservés

FORGE ne construit pas ce qui a été demandé. FORGE construit ce qui a été compris, spécifié, et rendu testable.